

Binary Search Problem :-

.. Given sorted array, target.

.. Input :- $arr[] = \{3, 4, 6, 7\}$, target = 4

Linear Search :- $O(n)$.

```
int binarySearch (int arr[], int left, int right, int target) {
```

```
    if while
```

```
        if (left > right)
```

```
            return -1;
```

```
        int mid = left + (right - left) / 2;
```

```
        if (arr[mid] == midtarget)
```

```
            return mid;
```


Dated:

else if (arr[mid] < target)
 return binarySearch(arr, mid+1, right, target);

mid+1 index now becomes left index.

else
 return binarySearch(arr, left, mid, target);

mid index now becomes right index.

Ex:-

0	1	2	3	4	5	6	7	8
1	3	4	6	7	8	10	13	14

$$\left\lfloor \frac{0+8}{2} \right\rfloor = 4$$

arr[mid] = 7

Case 1: target = 7

arr[mid] == target

simply return found @ index mid (4).

Case 2: target = 10

arr[mid] = 7

7 < 10

1	3	4	6	7	8	10	13	14
---	---	---	---	---	---	----	----	----

mid+1

search in this

8	10	13	14
---	----	----	----

$$\left\lfloor \frac{5+8}{2} \right\rfloor = 6.5 = 6$$

mid = 6

arr[6] = 10 found @ index 6.

Case 3: target = 5

0	1	2	3	4	5	6	7	8
1	3	4	6	7	8	10	13	14

7 > 5 so search left → mid.

1	3	4	6	7
---	---	---	---	---

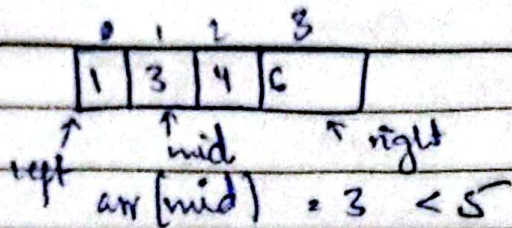
arr[mid] = 4 < 5 so search mid+1 → right

6	7
---	---

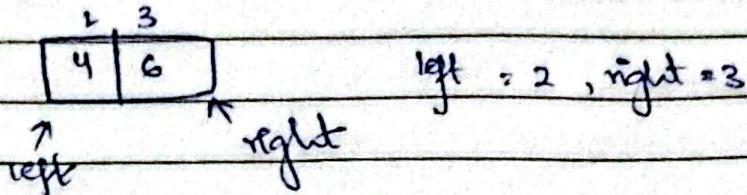
6 > 5 search in left → mid

$$\left\lfloor \frac{3+4}{2} \right\rfloor = 3$$

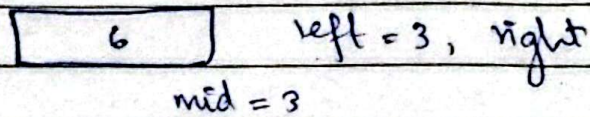
Dated:



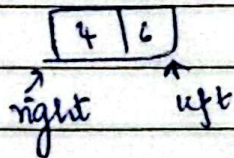
so search in $mid + 1$ to right



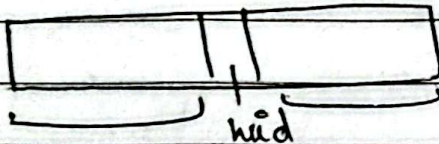
$mid = 2, arr[2] = 4 < 5$ search $mid + 1 \rightarrow right$
 $left \rightarrow mid$



$6 > 5$ search in left half
 $arr[3 \dots 2]$



$left > right$ return -1



left $mid + 1$ to right

- - when searching $left \rightarrow mid$, we do not include mid element itself. $left \rightarrow mid - 1$

Time Complexity :-

$$T(n) = 1 + T(n/2)$$

$$n/2^i = 1$$

$$n = 2^i$$

$$\log n = i \log_2 2$$

$$i = \log_2 n$$

$$\Rightarrow O(\log n)$$

Loop Invariant for Binary Search:-

Dated:

Three Conditions:-

- .. Array arr is sorted in ascending order.
- .. $left \leq right$
- .. target belongs to $arr[left \dots right]$

Initialization:-

- .. Array is sorted due to precondition of the method.
- .. since arr . length is atleast 1 so $left \leq right$.
- .. Precondition guarantees that target is in arr .

Maintenance:-

Case 1:- If $arr[mid] == target$

Case 2:- $arr[mid] > target$

$left \rightarrow mid - 1$ $arr[left \dots mid - 1]$ or $arr[left \dots mid]$
so $right = mid$ or $right = mid - 1$
subarray $arr[left \dots right]$ still contains target.

Case 3:- if $arr[mid] < target \rightarrow arr[mid + 1 \dots right]$

$left = mid + 1$

.. subarray $arr[left \dots right]$ still contains target.

After each iteration:-

- .. Array is still sorted (unchanged).
- .. New bounds still satisfy $left \leq right$
- .. if target exists its still in range $arr[left \dots right]$.

Termination:-

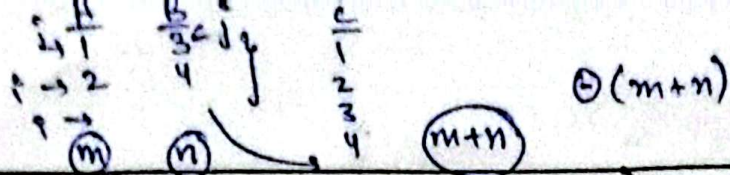
- .. $arr[mid] == target$ so return mid index.
- .. if $left > right$ so terminates if target doesnot exists in $arr[left \dots right]$.

DIVIDE N CONQUER : MERGE SORT

Iterative Merge sort:-

Compare $A[i]$ & $B[j]$

A	B	C (final sorted array)
$i \rightarrow 2$	$5 < 9$	2
$i \rightarrow 8$	$9 < 12$	5
$i \rightarrow 15$		8
		9
		12, 15



Dated:

Algorithm Merge (A, B, m, n)

```

{
    i = 1, j = 1, k = 1
    while (i <= m & j <= n)
    {
        if (A[i] < B[j])
            C[k++] = A[i++];
        else
            C[k++] = B[j++];
    }
    for (; i <= m; i++)
        C[k++] = A[i];
    for (; j <= n; j++)
        C[k++] = B[j];
}

```

M-way Merging

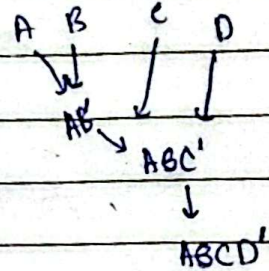
A B C D

merging 3 arrays

↳ 3-way merge

A B C D

Now if I want to merge 4 arrays using 2-way merge



Merge Sort Algorithm: Divide And Conquer:-

Alg. MergeSort (A, l, r)

if (l < r)

{

mid = (l + r) / 2;

T(n/2) - MergeSort (A, l, mid);

T(n/2) - MergeSort (mid + 1, r);

n - Merge (A, l, mid, r);

}

}

T(n) = 2T(n/2) + n

n log n

n = 1

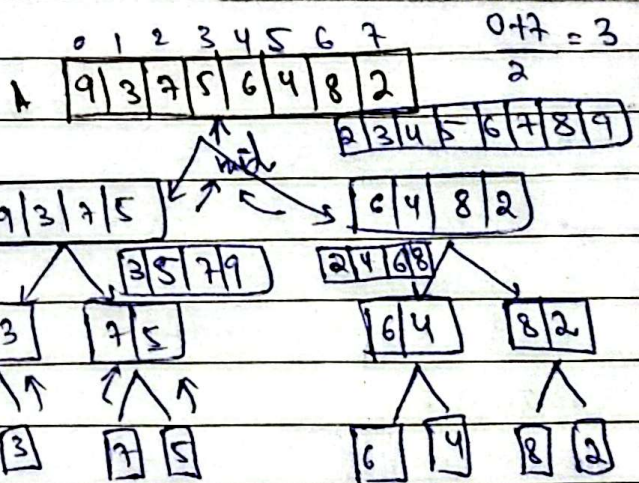
base case :- only one element left.

T(n) = { 1 n = 1
2T(n/2) + n n > 1

Master's theorem

a = 2, b = 2, d = 1

2 = 2' ⇒ O(n log n)



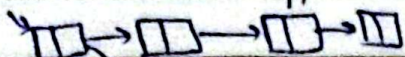
Dated:

Pros n Cons of Merge Sort:-

Pros:-

- large size list (works effectively on that).

• Linked list



• External sorting:



4GB



5GB 10GB

• Stable

Cons:-

- Extra space (not inplace sort)

- No small problem (for small sized lists it's slower).

- we use insertion sort (better for small sized arrays)

also stable.

stable algorithms	→ Insertion sort
	→ Merge sort
	→ Bubble sort.

- Recursive, recursive call (call stack)

Space ($n + \log n$)

$h = \log n$

extra arr for call stacks

Loop Invariant:-

merging

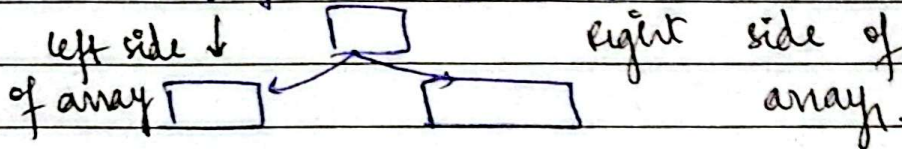
merge

left $\rightarrow$ mid sorted $\leftarrow$ mid+1 $\rightarrow$ right sorted

Initialization:- $i=1, j=1, k=1$ left subarray $arr[1 \dots m]$ right subarray $arr[m+1 \dots r]$

$temp[1, k-1]$ is empty. This empty subarray contains $k-1=0$ smallest elements of L and R since $i=1=j=1=k=1$.

$L[i], R[j]$ or $arr[left], arr[mid+1]$



Maintenance:-

- In each iteration, we compare $arr[i]$ (from left) and $arr[j]$ (from right).

- The smaller of two is appended to $temp[k]$.

- This guarantees that $temp[0 \dots k]$ contains the $k+1$ smallest elements of combined subarrays in sorted order.

Dated:

do $i++$ until it reaches m $i \leq m$

$j++ \rightarrow j \leq r \rightarrow \text{right}$

Termination:-

- The loop ends when one subarray is fully scanned.
- At this point, $\text{temp}[0..k-1]$ contains the smallest k elements in sorted order.
- The remaining elements from unfinished subarray are greater than or equal to the ones already copied so they can simply be appended.
- Finally $\text{arr}[l..r]$ is overwritten with temp making it sorted.

Loop Invariant for merge sort function

Recursive structure:-

- $\text{mergeSort}(\text{arr}, l, m)$ sorts $\text{arr}[l..m]$
- $\text{mergeSort}(\text{arr}, m+1, r)$ sorts $\text{arr}[m+1..r]$
- Then $\text{merge}(\text{arr}, l, m, r)$ merges them.

At the beginning of each call to $\text{merge}(\text{arr}, l, m, r)$, the subarrays $\text{arr}[l..m]$

Merge sort merge function code:-

```
void merge (int arr[], int l, int m, int r)
```

```
{ int i = l; int j = m+1; int k = 0;
```

```
  int temp[r-l+1];
```

```
  while (i <= m && j <= r) {
```

```
    if (arr[i] <= arr[j])
```

```
      temp[k++] = arr[i++];
```

```
    else
```

```
      temp[k++] = arr[j++];
```

```
  } while (i <= m)
```

```
    temp[k++] = arr[i++];
```

```
  while (j <= r)
```

```
    temp[k++] = arr[j++];
```

```
  for (int p = 0; p < k; p++)
```

```
    arr[p] = temp[p];
```

```
}
```

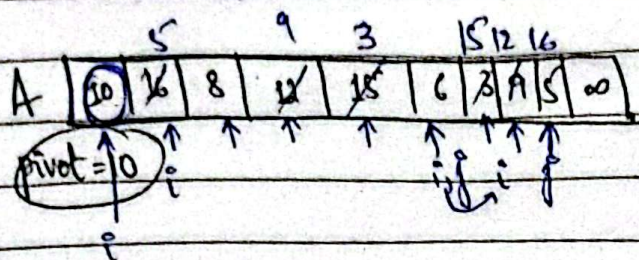
```
}
```


Dated:

Quicksort:-

6 3 5 4 2 1 7 ^{this is sorted}
 we're sure since we've picked a smaller element
 than the rest.

An element is in sorted position if elements right to it are greater & elements left to it are lesser.



- Start i from pivot and j from infinity.
- i will look for elements greater than pivot and j will search for elements smaller than pivot element.
- Then exchange its positions.
- increment i until larger element than pivot found and decrement j until smaller element than pivot found then swap.
- swap pivot element & A[j].
- Now that pivot element is sorted.

Partition (l, h)

```

i
pivot = A[l];
while (i < j)
{
  do {
    i++;
  } while (A[i] <= pivot);
  do {
    j--;
  } while (A[j] > pivot);
  swap(A[i], A[j]);
}
swap(A[l], A[j]);
return j;

```

Quicksort (l, h)

```

if (l < h)
{
  j = partition(l, h);
  Quicksort(l, j);
  Quicksort(j+1, h);
}

```


Dated:

```
Algorithm Quicksort (arr, start, end) :  
    if start < end :  
        pivotIndex = partition(arr, start, end);  
        quicksort (arr, start, pivotIndex-1);  
        quicksort (arr, pivotIndex+1, end);
```

```
Algorithm Partition (arr, start, end) {  
    mid = start + (end - start) / 2;  
    pivot = arr[mid]  
    swap (arr[pivot], arr[start]);  
    i = start + 1;  
    j = end
```

```
    while True {  
        while i <= j and arr[i] <= pivot {  
            i++; }  
        while i <= j and arr[j] > pivot {  
            j--; }  
        if i > j {  
            break;  
        }  
        swap (arr[i], arr[j]);  
    }
```

```
    swap (arr[start], arr[j]);  
    return j;  
}
```

Algorithms	Worst Case	Best Case
Insertion sort	$O(n^2)$	$O(n)$
Merge sort	$O(n \log n)$	$O(n \log n)$
Heap sort	$O(n \log n)$	$O(n \log n)$
Quick sort	$O(n^2)$	$O(n \log n)$
Bubble sort	$O(n^2)$	$O(n)$

Q1) a) Master's Theorem:-

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$a \geq 1, b \geq 2, c > 0$$

$$f(n) \in \Theta(n^d) \quad d \geq 0$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Case 4:- $f(n) \in \Theta(n^{\log_b a} \log^k n)$

$$T(n) \in \Theta(n^{\log_b a} \log^{k+1} n)$$

a. $T(n) = 100T\left(\frac{n}{10}\right) + n^2 \log n + n^2 + 1$

$$a = 100, b = 10 \quad f(n) \in \Theta(n^{\log_b a} \log^k n)$$

$$n^2 \log^1 n$$

$$\log_{10} 100 = 2$$

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n) = \Theta(n^2 \log^2 n)$$

b) $T(n) = 2T\left(\frac{n}{4}\right) + 3T\left(\frac{n}{2}\right) + n$

It has two recurrence branches so we cannot apply Master's theorem.

c) $T(n) = 3T\left(\frac{n}{3}\right) + \underbrace{\sqrt{n}}_{n^{\frac{1}{2}}} + 1$

$$a = 3, b = 3, d = \frac{1}{2}$$

$$3 > 3^{\frac{1}{2}}$$

$$a > b^d$$

$$T(n) = \Theta(\log^a) = \Theta(n^{\log_b a}) = \Theta(n^{\log_3 3})$$

Substitution Method:-

$$T(n) = 3T\left(\frac{n}{2}\right) + n^2 \quad ; \text{ apply for } \underline{a} \quad T(n) = O(n^2)$$

$$T(n) = O(n^2) = cn^2 \rightarrow n = \frac{n}{2} \quad c\left(\frac{n}{2}\right)^2 = \frac{cn^2}{4}$$

$$T(n) = 3c\left(\frac{n^2}{4}\right) + n^2$$

$$cn^2 = \frac{3cn^2}{4} + n^2$$

$$\frac{cn^2}{4} = n^2$$

$$c = 4, c > 0 \text{ correct.}$$

$$T(n) = O(n \log n) \quad T(n/2) = O(n/2 \log n/2)$$

$$cn \log n = 3c \frac{n}{2} \log \frac{n}{2} + n^2$$

$$cn \log n = 3c \left(\frac{n}{2} (\log n - \log 2) \right) + n^2$$

$$\log \frac{a}{b} = \log a - \log b$$

$$cn \log n = 3c \left(\frac{n}{2} (\log n - 1) \right) + n^2$$

$$= \frac{3c}{2} n \log n - \frac{3c}{2} n + n^2$$

n^2 term dominates for large n .

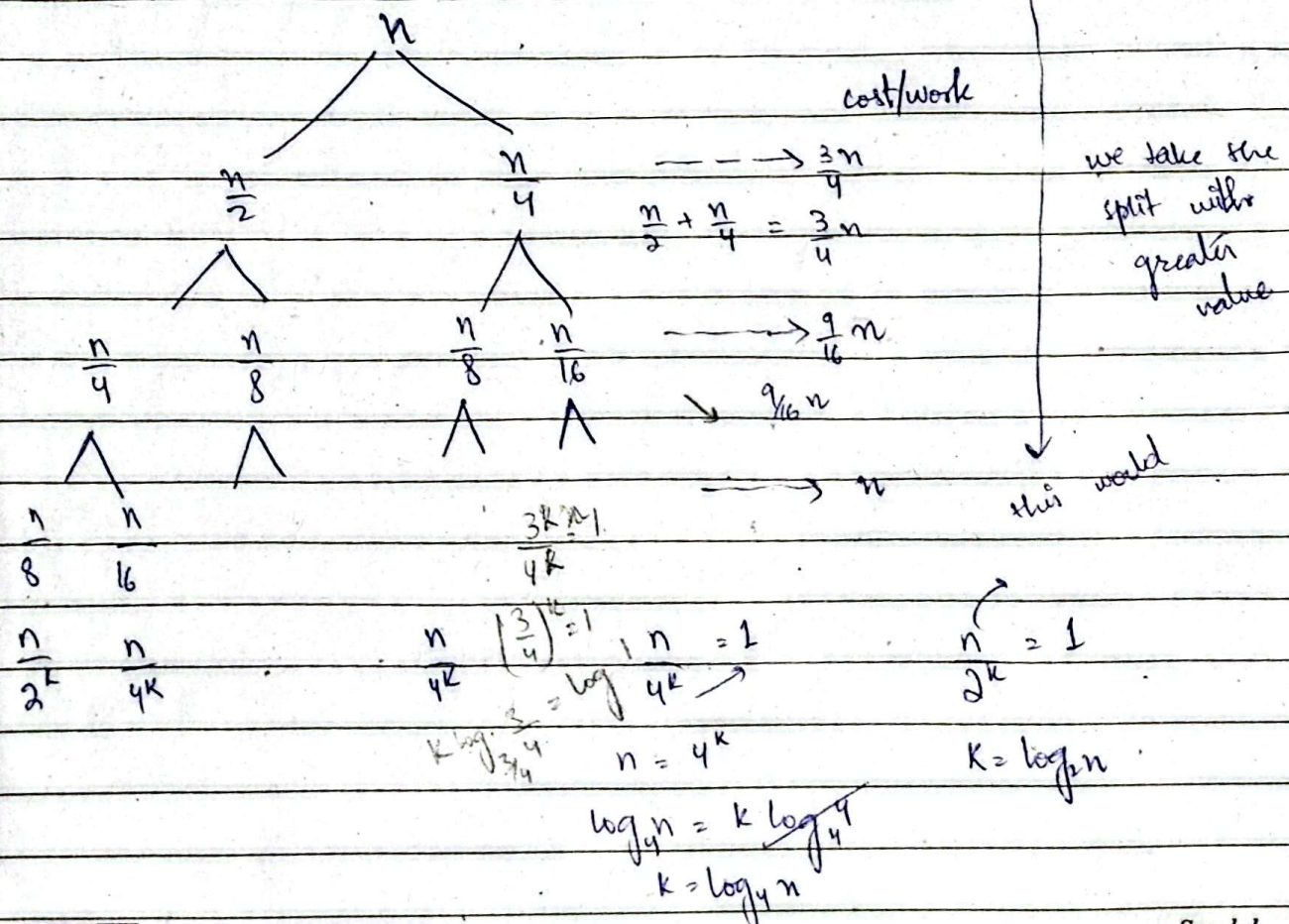
$$\times T(n) = O(n \log n)$$

Incorrect!

By Recursive Tree or Iterative Method.

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + n$$

$$\frac{1}{2} > \frac{1}{4}$$



Total cost $n \log_2 n$ $\therefore$ cost at level 0: n

$$\therefore \text{cost at level 1: } \frac{n}{2} + \frac{n}{2} = \frac{3n}{4}$$

$$\therefore \text{cost at level 2: } \frac{n}{4} + \frac{n}{8} + \frac{n}{8} + \frac{n}{16} = \frac{9n}{16}$$

 n $\frac{3n}{4}$ $\frac{9n}{16}$

$$\text{cost at level } i: \left(\frac{3}{4}\right)^i n$$

$$T(n) = n + \frac{3n}{4} + \left(\frac{3}{4}\right)^2 n$$

$$\text{Common Ratio} = \frac{T_2}{T_1}$$

$$r = 3/4$$

$$= \frac{3/4}{1} = 3/4$$

$$S_{\infty} = n \left(\sum_{i=0}^{\infty} \frac{1}{1-r} \right)$$

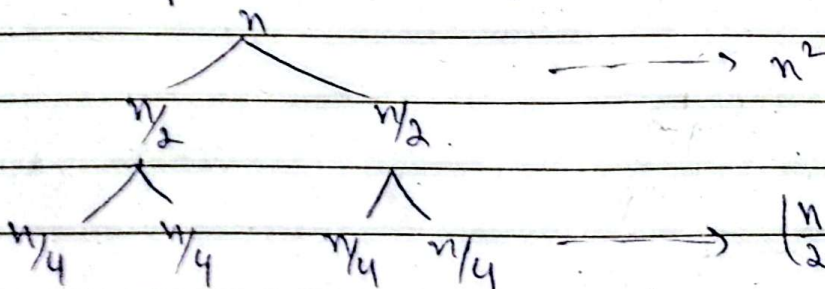
$$S_{\infty} = n \left(\frac{1}{1-3/4} \right) = 4n$$

$$\text{total cost} = 4n$$

$$T(n) = O(4n) = O(n)$$

$$b) T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

split into two parts



$$\left(\frac{n}{2}\right)^2 + \left(\frac{n}{2}\right)^2 = \frac{n^2}{2}$$

$$4 \cdot \left(\frac{n}{4}\right)^2 = \frac{n^2}{4}$$

$$\text{Cost} = 1n^2 + \frac{1}{2}n^2 + \frac{1}{4}n^2 + \dots$$

$$S_{\infty} = n^2 \left(\sum_{i=0}^{\infty} \frac{1}{1-n} \right)$$

$$S_{\infty} = n^2 \left(\frac{1}{1-\frac{1}{2}} \right)$$

$$\text{total cost} = 2n^2$$

$$\Rightarrow O(2n^2) = O(n^2)$$

Loop invariant :- Sum Algorithm

Initialization :- $\text{sum} = 0, i = 0$, invariant holds because sum is 0 for first 0 elements of the array.

Maintenance :- $i = i + 1$

sum is the sum for $A[0..i]$

Termination :-

terminates when $i = n$.

Q2) Proof. $1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < 3^n < \dots < n^n$

$$(1^o) \quad n^3 + 2^n = O(2^n)$$

$$n^3 + 2^n \leq C \cdot 2^n$$

$$n^3 \leq C \cdot 2^n - 2^n$$

$$n^3 \leq (C-1) 2^n$$

choose $C = 2$

for large n , choose $C = 2$

$$n^3 \leq 2^n$$

$$n^3 = 2^n$$

$$= \frac{n \log 2}{\log 2}$$

$$n^3 = 2^n \Rightarrow e^{n \ln 2} = n^3$$

$$a^x = e^{x \ln a}$$

$$n \ln 2 \ln e = 3 \ln n$$

$$\ln n = \frac{\ln 2}{3} n$$

3

$$n e^{-(\ln 2/3)n} = 1$$

Multiply by $-\frac{\ln 2}{3}$

$$\left(-\frac{\ln 2}{3} n\right) e^{-\frac{\ln 2}{3} n} = -\frac{\ln 2}{3}$$

$$y = -\frac{\ln 2}{3} n, \quad y e^y = -\frac{\ln 2}{3}$$

$$y = W_k\left(-\frac{\ln 2}{3}\right)$$

$$n = \frac{-3}{\ln 2} W_k\left(-\frac{\ln 2}{3}\right)$$

$$-\frac{\ln 2}{3} \in (-1/e, 0)$$

$$n \approx 1.37, \quad n \approx 9.9 \Rightarrow \boxed{10}$$

for all $n \geq 10$, this holds true.

$$\frac{n^2 + 4}{2n^2 + 3n + 1} = \Theta(1)$$

$$c_1 \leq \frac{n^2 + 4}{2n^2 + 3n + 1} \leq c_2$$

$$\begin{aligned} &\downarrow \quad n \rightarrow \infty \\ &\approx \frac{n^2}{2n^2} = \frac{1}{2} \quad \begin{aligned} &\cancel{n^2} \left(1 + 4/n^2\right) \\ &\cancel{n^2} \left(2 + 3/n + 1/n^2\right) \end{aligned} \\ &\frac{n^2}{2n^2} \leq \frac{1}{2} \quad \begin{aligned} &1+0 > \boxed{\frac{1}{2}} \\ &2+0+0 \end{aligned} \end{aligned}$$

$$n^2 \leq n^2 + 4 \leq 5n^2, \quad 2n^2 \leq 2n^2 + 3n + 1 \leq 6n^2$$

Divide the inequalities:-

$$\frac{n^2}{6n^2} \leq \frac{n^2 + 4}{2n^2 + 3n + 1} \leq \frac{5n^2}{2n^2}$$

so for all $n \geq 1$

$$\boxed{\frac{1}{6} < \frac{n^2 + 4}{2n^2 + 3n + 1} \leq \frac{5}{2}} = \Theta(1)$$

Date: _____

$$\frac{n^2 + 4}{2n^2 + 3n + 1} = \Theta(1)$$

$$n^2 \log n = \Theta(n^2)$$

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$
$$f(n) \leq c_2 \cdot n^2$$

$$\downarrow$$
$$\cancel{n^2} \log n \leq c_2 \cdot \cancel{n^2}$$

∴ No constant can keep $\log n \leq c_2$ for large n .

$$\boxed{n^2 \log n \neq \Theta(n^2)}$$

$$T(n) = \begin{cases} 1 & n=0 \\ 2T(n-1)+1 & n>0 \end{cases}$$

Iterative Substitution

$$2T(n-1)+1$$

$$2[2T(n-1-1)]+1$$

$$4T(n-2)+2+1$$

$$4T(n-2)+3$$

$$4(2T(n-2-1)+1)+3$$

$$8T(n-3)+4+3$$

$$8T(n-3)+7$$

$$2^3T(n-3)+2^2+2+1$$

$$2^k T(n-k) + 2^{k-1} + \dots + 2^0$$

* Geometric series formula.

$$1+r+r^2+\dots+r^{k-1} = \frac{r^k-1}{r-1}$$

$$1+2+2^2+\dots+2^{k-1}$$

$$= \frac{2^k-1}{2-1} = 2^k-1$$

$$T(n)=1 \text{ when } n=0$$

$$T(0)=1$$

$$n-k=0$$

$$n=k$$

$$2^n T(0) + 1 + 2 + \dots + 2^{k-1}$$

$$2^n(1) + 2^k - 1$$

$$2^n + 2^n - 1$$

$$\underbrace{\quad}$$

$$a+a=2a$$

$$2^n+2^n=2(2^n)$$

$$2 \cdot 2^n - 1$$

$$T(n)=2^{n+1}-1$$

$$T(n) = 2^{n+1} - 1 = O(2^n)$$

$$T(n) = \begin{cases} 1 & n=1 \\ \sqrt{2} T\left(\frac{n}{2}\right) + \sqrt{n} & n>1 \end{cases}$$

Proof for $O(n \log n)$ and for $O(1)$.

$$c n \log n \rightarrow c \frac{n}{2} \log \frac{n}{2}$$

$n \rightarrow n/2$

$$\sqrt{2} T\left(\frac{n}{2}\right) + \sqrt{n}$$

$$\sqrt{2} \left(\sqrt{2} T\left(\frac{n}{4}\right) + \sqrt{\frac{n}{2}} \right) + \sqrt{n}$$

$$2 T\left(\frac{n}{4}\right) + \sqrt{2} \sqrt{\frac{n}{2}} + \sqrt{n}$$

$$\cancel{\sqrt{2}} \times \frac{\sqrt{n}}{\cancel{\sqrt{2}}}$$

$$2 T\left(\frac{n}{4}\right) + \sqrt{n} + \sqrt{n} \quad 2 T\left(\frac{n}{4}\right) + 2\sqrt{n}$$

$$\Rightarrow 2 \left(\sqrt{2} T\left(\frac{n}{8}\right) + \sqrt{\frac{n}{4}} \right) + \sqrt{n} + \sqrt{n}$$

$$2\sqrt{2} T\left(\frac{n}{8}\right) + 2\sqrt{\frac{n}{4}} + 2\sqrt{n}$$

$$2\sqrt{2} T\left(\frac{n}{8}\right) + \cancel{2} \frac{\sqrt{n}}{\cancel{2}} + 2\sqrt{n}$$

$$2\sqrt{2} T\left(\frac{n}{8}\right) + 3\sqrt{n}$$

$$\left(\sqrt{2}\right)^i T\left(\frac{n}{2^i}\right) + i\sqrt{n}$$

can also be written

$$\frac{n}{2^i} = 1$$

as:-

$$n = 2^i$$

$$\sqrt{2^i}$$

$$2^{i/2} = \sqrt{n}$$

$$\log_2 n = i \log_2 2$$

$i = \log_2 n$

$$\sqrt{n}$$

$$= O(\sqrt{n} \log n)$$

$O(n \log n)$ is True.

$O(1)$ is False.

array → sorted

```

int left = 0, int right = n-1
while (left <= right) {
    int mid = left + (right - left) / 2
    if (arr[mid] < t)
        left = mid + 1;
    else
        right = mid - 1;
}

```

```

int pos = left;

```

```

left = pos - 1
right = pos
vector<int> result;

```

```

while (k > 0) {
    if (left < 0) {
        result.push_back(arr[right]);
        right++;
    }
}

```

```

else if (right >= n) {
    result.push_back(arr[left]);
    left--;
}

```

```

else {
    if (abs(arr[left] - t) <= abs(arr[right] - t)) {
        result.push_back(arr[left]);
        left--;
    } else {

```

```

        result.push_back(arr[right]);
        right++;
    }
}

```


Arr = [2, 9, 3, 8, 11, 1, 6, 6], n = 8

split

$$11 - 2 = 9$$

[2, 9, 3, 8]

[11, 1, 6, 6]

[2, 9]

[3, 8]

[11, 1]

[6, 6]

$$\text{min} = 2$$

$$\text{min} = 3$$

$$\text{min} = 1$$

$$6 - 6 = 0$$

$$\text{max} = 9$$

$$\text{max} = 8$$

$$\text{max} = 11$$

$$\text{profit} = 7$$

$$\text{profit} = 5$$

$$\text{profit} = 10$$

but 11 comes first

MaxProfit (A, low, high)

if low == high.

return 0, low, low // profit = 0, buy day = sell day

$$\text{mid} = (\text{low} + \text{high}) / 2;$$

leftProfit, leftBuy, leftSell = MaxProfit (A, low, mid)

rightProfit, rightBuy, rightSell = MaxProfit (A, mid + 1, high)

minLeftPrice = min(A[low..mid])

minLeftDay = day of minLeftPrice

$\text{maxRightPrice} = \max(A[\text{mid}+1 \dots \text{high}])$
 $\text{maxRightDay} = \text{day of maxRightPrice}$
 $\text{crossProfit} = \text{maxRightPrice} - \text{minLeftPrice}$

if $\text{crossProfit} > 0$:

$\text{crossBuy}, \text{crossSell} = \text{minLeftDay}, \text{maxRightDay}$

else:

$\text{crossProfit} = 0$

$\text{crossBuy} = \text{crossSell} = -1$

if $\text{leftProfit} \geq \text{rightProfit}$ and $\text{leftProfit} \geq \text{crossProfit}$:
return $\text{leftProfit}, \text{leftBuy}, \text{leftSell}$

elif $\text{rightProfit} \geq \text{leftProfit}$ and $\text{rightProfit} \geq \text{crossProfit}$:
return $\text{rightProfit}, \text{rightBuy}, \text{rightSell}$

else:

return $\text{crossProfit}, \text{crossBuy}, \text{crossSell}$

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$